

GAT vs. the Local-Attention Transformer

The same one-hop reach, with very different bills

src/layers.py — GATConv and GlobalAttnConv(local: true) · 2026-07-06

In one sentence. GAT and the local-attention transformer restrict a node to its graph neighbours in exactly the same way — the difference is that GAT only ever **computes** the edge entries (sparse, $O(E)$), while `local: true` computes the **full dense** $N \times N$ attention matrix and then throws away everything that is not an edge — paying the whole transformer bill, plus a masking surcharge, to use $\sim 0.4\%$ of what it bought.

1 Why the repo has both

`local: true` was not built to be a good model. It exists for one controlled experiment (CHANGELOG 2026-06-25): the global-attention transformer is **not** depth-bounded — one all-pairs layer reaches any node, so the 3^L capacity wall of Ye et al. never binds and the data-lever experiment cannot engage. To make the wall appear, we needed the **same** model — same scaled dot-product attention, same W_Q, W_K, W_V parameterisation, same softmax, same parameter count — with only the **reach** changed from all-pairs to one hop per layer. That is `GlobalAttnConv(local: true)`: the minimal-diff, depth-bounded transformer.

GAT would not have been that experiment. It restricts reach the same way, but it also changes the attention **mechanism** (Section 2), so any behavioural difference would be confounded. The price of the clean comparison is efficiency — and the price is steep (Section 3).

2 The two mechanisms

Both compute, for node i , a weighted average over the **same** set — the neighbours $N(i) \cup \{i\}$. They differ in how scores are produced and, crucially, over what domain they are computed.

2.1 GAT: additive attention, computed only on edges

GATConv (Veličković et al. 2018) is a PyG `MessagePassing` module. One shared projection W , and a learned **additive** score per edge:

$$e_{ij} = \text{LeakyReLU}(a_{\text{src}}^\top W h_j + a_{\text{dst}}^\top W h_i), \quad \alpha_{ij} = \text{softmax}_{j \in N(i) \cup \{i\}}(e_{ij})$$

The implementation never sees a non-edge: scores live in a tensor of shape $[E, H]$ (one row per edge), the softmax is a **segmented** softmax grouped by destination node (`pyg_softmax`), and aggregation is a scatter-add along edges. Work and memory are $O(E \cdot d)$ — for our caterpillars ($E \approx 2n$), **linear in nodes**.

2.2 Local attention: dot-product attention, computed on all pairs, then masked

`GlobalAttnConv` is a transformer layer over the node set: separate W_Q, W_K, W_V , scaled dot-product scores for **every pair in the batch**:

$$A_{ij} = \frac{Q_i \cdot K_j}{\sqrt{d_h}} \quad \text{for all } i, j \in [N]_{\text{batch}}$$

— an `einsum` producing a dense $[N, N, H]$ tensor where N is the **total** node count of the batch (32 graphs $\times$ 24 nodes = 768), followed by a cross-graph mask. With `local: true` it then builds a dense boolean adjacency $[N, N]$ from `edge_index`, masks every non-neighbour entry to $-\infty$, and softmaxes the full matrix:

$$A_{ij} \leftarrow \begin{cases} A_{ij} & \text{if } (j, i) \in E \text{ or } i = j \\ -\infty & \text{otherwise} \end{cases}$$

The **used** entries after masking are exactly GAT’s domain. Everything else was computed, written to memory, masked, exponentiated to 0 — and discarded.

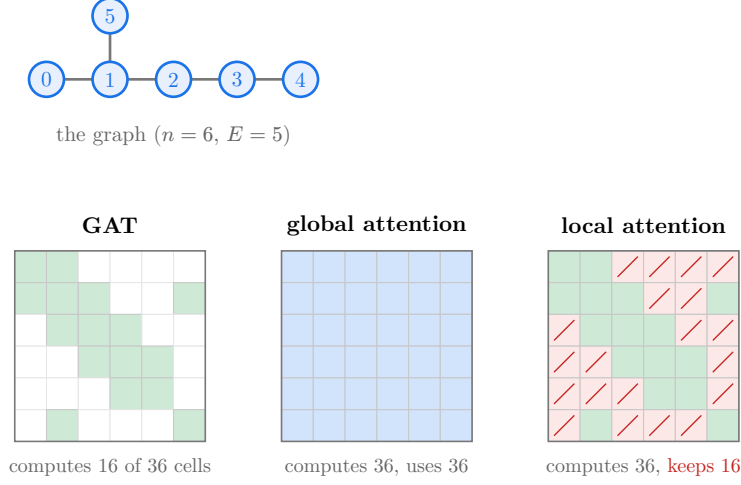


Figure 1: Score computation per layer on a toy 6-node graph (self-loops included). **green** cells are neighbour scores — the only ones any of the three models ultimately **uses** for the local models. GAT (left) never materialises anything else. Local attention (right) materialises the full matrix like the global model (middle), then **masks out** every non-edge. At batch scale ($N = 768$) the kept fraction drops from 16/36 to $\sim 0.4\%$.

3 Why local attention is the slowest of the three

Measured on identical runs (conn_diameter_controlled, depth 2, hidden 64, batch 32, $n = 24$, Apple MPS):

layer	train, s/epoch	inference, ms/graph	score domain
GAT (gat)	3.3 – 7.4	0.44 – 0.66	$[E, H]$: edges only
global (global_attn)	3.6 – 4.8	1.60 – 1.96	$[N, N, H]$: all pairs, all used
local (local: true)	20.7	2.50	$[N, N, H]$ plus dense mask, 0.4% used

Table 1: Wall-clock from results/*.json timing blocks (runs of 2026-06-25 – 07-02). Local attention is $\sim 3\text{--}6\times$ slower per epoch than GAT and — the telling part — $\sim 5\times$ slower than the **global** model it strictly subsets.

Three compounding reasons, in decreasing order of principle:

3.1 It pays the full all-pairs bill for a sparse result

Per layer, per forward, the batch has $N = 768$ nodes and $E + N \approx 2,368$ neighbour-or-self pairs. The dense path computes $N^2 = 589,824$ pair scores $\times 4$ heads — then keeps 0.4% of them. GAT’s compute **is** the kept set. Asymptotically: $O(N^2d)$ vs $O(Ed)$ per layer, a factor of $N^2/E \approx 250$ in score work on these sparse graphs. (Why doesn’t the **global** model already suffer? It does the same N^2 work — but as one dense matmul, the operation GPUs are best at, and every computed entry is used. That is Table 1’s second row costing only $\sim 3\times$ GAT’s inference. Dense hardware efficiency hides the asymptotic waste at $n = 24$; it stops hiding around the point where $[N, N, H]$ stops fitting in cache — and would be fatal at Yehudai’s $n = 50$ or beyond.)

3.2 The mask itself is a second $O(N^2)$ pass — in eager mode, several

`local: true` adds, on top of the global layer’s work, **per layer per step**: a fresh $[N, N]$ boolean adjacency allocation, two advanced-indexing writes into it, a `masked_fill` over $[N, N, H]$, and a softmax whose rows are mostly $-\infty$. None of these fuse: each is a separate memory-bound kernel pass over the ~ 9.4 MB score tensor (vs GAT’s ~ 37 KB edge tensor — a $250\times$ memory-traffic gap). On MPS specifically, boolean advanced indexing is a notoriously slow path (extra synchronisation, occasional CPU fallback), which is the best explanation for local costing $5\times$ the global model when it “only” adds masking. The einsum-based attention also forgoes the fused scaled-dot-product kernel — a deliberate trade for the `store_attn` inspection hook, which needs the explicit $[N, N, H]$ tensor anyway.

3.3 The waste grows with the batch, not the graph

N is the batch-total node count, so doubling `batch_size` quadruples the dense work per step while the useful (edge) work merely doubles. GAT’s cost is batch-linear. The gap in Table 1 is at batch 32; larger batches widen it quadratically.

Rule of thumb. Dense masked attention costs $O(N^2)$ no matter how sparse the mask; sparsity only pays when it is **structural** (never compute the entry) rather than **post-hoc** (compute, then hide). GAT is structural sparsity; `local: true` is post-hoc sparsity. The scratchpad-CoT attention masks in `transformer.py` are post-hoc too — fine there, because those sequences genuinely need most pairs.

4 What is actually different, mechanism-wise

Reach is identical (one hop per layer, self included). Everything else differs slightly — which is exactly why `local: true` and not GAT is the controlled contrast to the global transformer:

	GAT	local attention
score form	additive: $a^\top [Wh_i \parallel Wh_j]$ + LeakyReLU	scaled dot-product: $Q_i \cdot K_j / \sqrt{d_h}$
projections	one shared W ; per-head vectors a	separate W_Q, W_K, W_V
softmax	segmented, over incoming edges	dense rows, non-edges at $-\infty$
compute/memory	$O(Ed) / [E, H]$	$O(N^2d) / [N, N, H] + [N, N]$ mask
same as global model?	no — different mechanism	yes — identical up to the mask
role in the repo	efficient depth-bounded baseline	controlled reach-ablation of the transformer

The additive-vs-dot-product distinction is not cosmetic: dot-product scores are bilinear in the two endpoints (content-content matching), while GAT’s additive form factorises into a source term plus a destination term — a strictly weaker interaction that, e.g., cannot express “attend to nodes whose key equals my query” without help from W . For connectivity on our datasets both were expressive enough; the choice never mattered for accuracy, only for the experimental logic.

5 Practical guidance

- **Need a depth-bounded model that trains fast?** GAT (or GIN/GCN). Batch-linear, and on MPS it is the cheapest attention we have.
- **Need the capacity-wall / reach-ablation comparison?** `local: true` — it is the only variant where “the same transformer, one hop per layer” is literally true. Accept the $\sim 5\times$ cost, keep batches small, and don’t run it at n much beyond 50 — the $[N, N, H]$ tensor and the mask passes scale quadratically in batch-total nodes.
- **If local attention ever becomes a workhorse** rather than an ablation: implement it as a `MessagePassing` module with dot-product scores (i.e., GAT’s plumbing with `GlobalAttnConv`’s

scoring). That keeps the mechanism-controlled comparison **and** the $O(E)$ bill — the two are only coupled in the current implementation, not in principle.